



Sandya Mannarswamy

Welcome to another installment of CodeSport. In this month's column, we'll explore the best lower bounds of algorithms to determine whether a given graph is connected or not. We will then discuss the problem of finding the minimum element in a circular sorted linked list, given an arbitrary pointer into the list.

Thanks to all the readers who commented on the problems we discussed in last month's column. Last month's takeaway question was to consider the well-known problem of deciding whether a given graph with N nodes is connected, and determine its best lower bound. The only question the algorithm can ask the adversary is of the form, "Does an edge exist between Vertex u and Vertex v ?" The readers were asked to come up with the best lower bound they could establish for this algorithm, using an adversary argument.

Since none of the solutions I received for this problem were completely correct, I am going to keep this problem open to readers this month also. However, in order to help those who were pretty close to getting the correct answer, I will give you some clues. Before trying to determine the lower bound for the 'graph connectedness' problem, let us first try to solve the problem using an algorithm known to us. Many of the questions on graphs, such as graph connectedness, or the presence of a cycle or path between 2 vertices, can be solved by using a variant of the 'depth first search traversal', which we have discussed in an earlier column.

How can you use depth first search to determine graph connectedness?

As we know, the depth first traversal is a graph traversal wherein we start visiting the children at each node, following any children that may exist at the current node, till we hit a leaf node. We then traverse back up one level, and restart the visit from there. This is unlike a breadth first search where nodes at each level are visited fully before any nodes at the next level are visited. For each node in the graph, perform a DFS from that node which is marked as 'root' for the current traversal iteration. If all other nodes of the graph cannot be visited during the depth first search

from the current root, then the graph is not connected. What is the complexity of such an algorithm? We know that DFS has a complexity of $O(V+E)$, where V is the number of vertices of the graph and E is the number of edges of the graph. Since E can be of the order of $O(V^2)$, the complexity is $O(V^2)$.

Lower bound for graph connectedness using adversary argument

Coming back to the question of the best lower bound for graph connectedness using an adversary argument, remember that if we do not probe all the possible edges of the graph using the question, "Is Vertex v adjacent to Vertex u ?" it is possible for the adversary to answer all our questions correctly and still leave Vertex u totally disconnected from Vertex v , resulting in the graph being disconnected. Therefore, any algorithm that gives the correct answer, must probe edges between each pair of vertices in the graph. Given a graph with N vertices, what is the maximum number possible for the edges in an undirected graph? The complete graph on ' N ' vertices has the maximum number of edges and it is equal to NC_2 , since an edge connects 2 vertices, and the number of ways of choosing any two vertices out of N vertices is (NC_2) . Note that NC_2 stands for choosing a combination of 2 items out of N items and is given by the formula $(N*(N-1)/2)$. Hence, any algorithm to determine graph connectedness must examine all (NC_2) edges. Thus the lower bound of any graph connectedness algorithm cannot be less than (NC_2) . With this clue, the readers should be able to solve the question of the best lower bound for graph connectedness using the adversary argument.

The question this month

In this month's column, we will revisit number searching. You are given a circular list of ' n ' numbers and the numbers on the list are strictly increasing. Since it is a circular list, the end of the